

Topic 3.8

3.8: The Use of const

We can use the const declaration to indicate that a variable's value will not change throughout its lifetime:

```
const float pi = 3.14159;
```

Note that **const** variables cannot be used to specify array sizes:

```
#define SIZE 12
```

```
const    int size = SIZE;
int      good[SIZE]; /* This works. */
int      bad[size];  /* This causes compilation errors. */
```

We can have **const** pointers in several different ways:

```
int  x,y,z;

const int *p1=&x; // p1 is a pointer to a
                  // const int, i.e. that
                  // value that p1 points to
                  // cannot change. p1 can point
                  // at many different
                  // locations throughout
                  // its lifetime.

int * const p2=&y; // p2 is a const pointer
                  // to an int, i.e. p2 cannot
                  // point anywhere except y
                  // during its
                  // lifetime. p2 can be used
                  // to change the
                  // value it points to.

const int * const p3=&z; // p3 is a
                        // const pointer to a
                        // const int. p3 cannot
                        // point anywhere except
                        // at z, and the value it
                        // points to cannot be
                        // changed.
```

When a formal parameter that is a pointer is declared to be a **const**, the variable it points to cannot be modified:

```
void cannot_change_x( const float *x )
```

```
{
*x = *x - 3; /* Causes compiler error. */
}
```

The address of a `const` variable cannot be passed as an actual parameter to a function whose corresponding formal parameter is not a `const` pointer:

```
const float pi = 3.14159;

void change_it( float  * x)
{
    ...
}

float use_it( const float  *x )
{
    ...
}

change_it( &pi ); // Compiler won't allow this.
                  // It must make a
                  // conservative decision to
                  // prevent pi from being
                  // changed, because it
                  // doesn't KNOW that
                  // change_it WON'T change it.

cout << use_it( &pi); // OK! Compiler knows
                      // use_it CAN'T change the
                      // value of pi
```

Through appropriate use of `const` pointers, we can specify tight requirements regarding what functions will do with their formal parameters. This allows us to achieve safety and efficiency simultaneously, passing pointers instead of large objects. This is the most important use of `const`.

When a member function is not going to modify member data, we can specify that the member function is `const`:

```
class Distance
{
    ...
    Distance operator+( int rhs ) const;
    ...
};

Distance Distance::operator + ( int rhs ) const
{
    return *this + Distance(rhs,0.0);
}
```

Topic 3.9 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-3-dot-9>)